

Traffic Sign Recognition Using SVM and MLP

Joshua Kao
Computer Systems Engineering
University OF Auckland
Auckland , New Zealand

I. INTRODUCTION

Traffic sign recognition is a multiclass image classification problem where a model is required to assign an image of a traffic sign to its correct label. This project investigates traffic sign recognition using the public Traffic Sign Classification dataset provided by a public Kaggle database [1], containing 43 categories of signs. This project compares two supervised learning methods: a multilayer perceptron (MLP) and a support vector machine (SVM). Models were trained validated as well as evaluated using the same preprocessed image data, and performances were compared using accuracy precision , recall and F-1 scores. The objective is to compare the strengths and limitations of each model and investigate the effects of preprocessing, model configuration and hyperparameter selection can influence each's ability to generalize to unseen traffic images.

II. METHODOLOGY

The main workflow contains four main stages: Data preparation, Model Selection, Model Training and Performance evaluation. Traffic sign images were first preprocessed into a common numerical representation and then divided into training, validation and test partitions through a group-aware split. Possible candidate MLP and SVM configurations were compared using only the training and validation data. When the best configurations for each model had been selected, the training and validation partitions were combined in which then the final models were retrained before being evaluated once on the previously unseen test set. Separating the model selection and the final testing procedure was used to reduce risk in selecting a model purely based on test set performance.

III. DATA PROCESSING

The original dataset contained 73,139 images

across 43 different classes of signs. Firstly using SHA-256 hashing, duplicate images were identified, leaving just 61,320 unique images files. Exact byte-identical duplicates were removed because they don't contribute new information and could give repeated observations disproportionate influence when training the model or evaluation as well as contribute to leakage between training and evaluating data. Removing them also reduced the computational cost of model training and hyperparameter turning. However, a distinction here should be made. Related frames belonging to the same traffic-sign sequence were retained because they were not byte-identical and contained legitimate differences in viewpoint, scale and illumination. These frames were then constrained to remain within the same training, validation or test partition during the group-aware split. For both MLP and SVM to receive inputs of the same format, we had to convert each of these images into a common numerical representation.

To accomplish this, every image was represented using 3 RGB channels. After that, each image was resized to 32 x 32 pixels and then normalized by dividing the pixel intensity by 255. The result of that was values in between 0 and 1 (RGB pixel intensities have a range of 0–255 thus dividing by 255 scales them to values in this range [0,1]). Normalizing also reduces the number of features whilst preserving their relative intensity and provides a consistent scale for both models. Each 32 x 32x 3 image was then flattened into a 3072-element feature vector allowing the same input representation to be used by both the MLP as well as the SVM models.

IV. DATA SPLITTING

To separate the process of developing the model hyper parameter tuning from final evaluation, the processed data set was then divided into training, validation and test partitions. A group-aware splitting strategy was used here meaning images belonging to the same traffic - sign sequence was restricted to remain in the same partitions. This reduces the possibility of leakage of similar frames from the same sign; prevents them

some appearing in training and other in evaluation datasets. Splitting was performed independently for each traffic-sign class to maintain class representation across all partitions. Initially, 20% of sequence groups were assigned to the test partition, while the remaining were divided into training and validation at a ratio of 80:20. This meant that theoretically the group proportions were split into 64% for training, 16% for validation and 20% for testing. However, since the number of images in the sequence group varies and we are applying a group-aware splitting strategy, the final split consisted of 32,010 training images (52%), 12,960 validation images (21.14%) and 16,350 test images (26.66%). The validation set generated was then used for hyperparameter and model selection while the test set was left alone until the model configurations were decided. After the model selection the training and validation sets were combined to provide 44,970 images (~73.3%) for final model fitting.

V. MLP MODEL DESIGN AND HYPERPARAMETER TUNING

In the multilayer perceptron model, aforementioned it receives each preprocessed image as a 3,072-element feature vector.

Three MLP configurations were selected to investigate the effects of network capacity/amount of neurons and learning rate without requiring an exhaustive hyperparameter search. An exhaustive search over all possible training parameters was not considered computationally practical for such large datasets where each image was represented by 3072 input features. Instead, we selected a targeted set of configurations to examine the effects of two important hyperparameters while keeping the computational cost reasonable. Having less hyperparameters to adjust also allows easier interpretation of results from direct changes in either hidden-layer width or learning rate. Thus our model should be considered the best performing configuration among the evaluated rather than finding a globally optimal MLP.

These three MLP configurations were determined from varying the amount of neurons in the hidden layer as well as the initial learning rate. Hidden-layer sizes of 64 and 128 were tested at learning rates of 0.003 and 0.001.

A 64-neuron hidden layer was used as a baseline, while increasing the width to 128 neurons roughly doubled the number of trainable parameters and therefore provided a comparison with a higher-capacity model. The first two configurations used the same learning rate of 0.003, which allows the effect of hidden-layer width / capacity to be seen. The third configuration uses 128 hidden neurons but with a reduced learning rate of 0.001, allowing the effect of smaller optimization steps to be observed independently of network width. For this experiment the MLP used the ReLU activation function with the Adam optimiser and a batch size of 128 as a golden standard. A fixed random seed of 306 was used to improve reproducibility between runs and the MLP was implemented using scikit-learn's MLPClassifier [2].

All models were allowed a maximum training iteration of 200, however it could end earlier if it reached convergence. Through increasing the number of hidden neurons, we expected to increase the representational capacity of the network. This would've allowed more complex relationships to be mapped between pixel features and the different traffic sign classes. However, this also increases computational cost and might also reduce generalization if the model becomes over tuned to the training data which would be detrimental. The learning rate controls the magnitude of parameter updates during optimization, which can be represented by this equation:

$$w_{t+1} = w_t - \eta \nabla L_t$$

η is the learning rate and L is the training loss.

$$Precision_c = \frac{TP_c}{TP_c + FP_c}$$

$$Recall_c = \frac{TP_c}{TP_c + FN_c}$$

$$F1_c = \frac{2Precision_c Recall_c}{Precision_c + Recall_c}$$

$$F1_{macro} = \frac{1}{43} \sum_{c=1}^{43} F1_c$$

$$Accuracy = \frac{\sum_{c=1}^{43} TP_c}{N}$$

Figure 1: Evaluation metrics used for model comparison.

This figure houses all metrics that will be used to analyze the information in the following tables. Accuracy represents the proportion of all test images that were classified correctly. Precision measures how often predictions for a given class were correct and is therefore affected by false positives. Recall measures show how many of the actual samples belonging to a class were successfully identified and is therefore affected by false negatives. F1-score is the harmonic means of precision and recall and provides a balanced measure when both types of error are important. Here macro averaging was used in order for each of the 43 traffic-sign classes to contribute equally to the precision, recall and F1 values.

We can observe that the model with 64 hidden neurons and a learning rate of 0.003 achieved the highest validation macro F1-score of 0.587 and accuracy of 0.5468. Increasing the layer size to 128 neurons at the same learning rate reduced the training loss from 0.216 to 0.121 however also reduced validation macro F1 to 0.584. Reducing the learning rate to 0.001 further decreased training loss to 0.0356 but produced a lower validation macro F1 of 0.579. As a result, improving fitting of the training data did not result in improved performance on unseen validation data. Based on the highest validation F1 score, the 64 neuron model with a learning rate of 0.003 was selected for final training.

Table 1: MLP validation performance for the three configurations tested

Hidden neurons	learning rate	iteration	loss	accuracy	Macro f1
64	0.003	79	0.217	0.547	0.588
128	0.003	89	0.121	0.531	0.584
128	0.001	152	0.036	0.518	0.580

VI. SVM MODEL DESIGN AND HYPERPARAMETER TUNING

The SVM model was tested using three kernel types - linear, radial basis function (RBF), and polynomial [3] with regularisation values $C = 0.1$, 1 and 10. This resulted in nine possible configurations. Candidate SVM configurations were screened using GridSearchCV [4]. The parameter C controls the trade-off between a wider decision margin and penalising classification errors. Smaller values of C allow greater tolerance for margin violations, whereas larger values place a greater penalty on misclassification.

Due to the computational cost of training SVMs on a large high - dimensional image dataset, hyperparameter selection was split into two stages. The first stage screened all nine configurations using 30% of the initial training groups. These contained 10,770 images from 204 groups. Three-fold StratifiedGroupKFold cross-validation was used here [5] to ensure related frames from the same traffic sign sequence remained in a single fold preventing leakage between training and validation data, and candidates were ranked using mean macro F1-score. For the RBF and polynomial candidates, $\gamma = \text{'scale'}$ was used, while the polynomial kernel used a degree of 3. Shown in table 2 are the three highest-ranked configurations all used a linear kernel, with $C = 0.1$ achieving the highest macro F1 of 0.315. This implies that additional non-linearity from the RBF and polynomial kernels did not improve screening performance for the chosen image representation.

Table 2: Stage-1 SVM Cross validation results

Kernel	params	Mean Macro F1	CV std deviation
linear	$C = 0.1$	0.316	0.00261
linear	$C = 1$	0.308	0.0174
linear	$C = 10$	0.306	0.0182
rbf	$C = 10$	0.266	0.0117
rbf	$C = 1$	0.242	0.00995
poly	$C = 10$	0.204	0.0165
poly	$C = 1$	0.190	0.0267

poly	$C = 0.1$	0.143091	0.030644
rbf	$C = 0.1$	0.052123	0.005275

Scores were ranked from highest to lowest scores (r.d 3 s.f)

As shown in table 2 the top two performing configurations were the linear SVMs with the parameter of values $C = 0.1$ and $C = 1$. Using the initial training partition, we then retrained and evaluated on the separate validation set. Although $C = 0.1$ ranked highest during preliminary screening, $C = 1$ achieved the higher full validation macro F1-score of 0.4912 compared with 0.4898 for $C = 0.1$ shown in Table 3. Though the difference was miniscule, however since $C = 1$ achieved the highest validation score, the linear SVM with $C = 1$ was selected for final model fitting.

Table 3: Stage-2 Validation of Two leading SVM Config

Params	Validation accuracy	Macro precision	Macro recall	Macro F1
Linear, $C = 0.1$	0.4911	0.5231	0.6700	0.4898
Linear, $C = 1$	0.4959	0.5166	0.6792	0.4912

(r.d 3 s.f)

VII. FINAL MODEL TRAINING AND EVALUATION

After hyperparameter selection, the training and validation partitions were combined to provide 44,970 images for final model fitting, while the 16,350-image test partition remained untouched. The MLP selected from our previous screening, uses a single hidden layer containing 64 neurons with an initial learning rate of 0.003. The maximum training iterations was set to 200 however the final model converges after 97 iterations, reaching a training loss of 0.185. The selected SVM used a linear kernel with ($C=1$). After, both models were then evaluated once using the test partition.

The MLP achieved an accuracy of 0.514, macro precision of 0.633, macro recall of 0.709 and macro F1-score of 0.575. In comparison, the SVM achieved an accuracy of 0.4467, macro precision of 0.5293, macro recall of 0.6795 and macro F1-score of 0.5037. The MLP therefore achieved an improvement of approximately 6.82 percentage in

accuracy and 7.15 percentage in macro F1. Because the dataset contained unequal numbers of examples across traffic-sign classes, macro averaged metrics were included alongside accuracy so that each class contributed equally to the comparison.

Table 4: Final test performance of the selected MLP and SVM

Model	Accuracy	Macro_precision	Macro recall	Macro f1
MLP	0.514	0.633	0.709	0.575
SVM	0.446	0.529	0.679	0.503

(r.d 3 s.f)

VIII. ANALYSIS AND DISCUSSION

Overall, the MLP produced stronger test performance than the SVM under the preprocessing and hyperparameter configurations. The MLP achieved approximately 6.82 percent higher accuracy and 7.15 percent higher macro F1-score. It also achieved higher macro precision and macro recall, indicating stronger average per-class performance across both measures. A reason for this difference could be the type of decision boundary each model represents. The selected SVM model had used a linear kernel and therefore separated classes using linear decision boundaries in the 3,072-dimensional pixel feature space. In comparison, the MLP used a nonlinear hidden layer, allowing it to learn more complex combinations of pixel features. This may have provided an advantage for traffic-sign images, where variations in illumination, scale and viewpoint can create nonlinear relationships between raw pixel values and class labels. However, these results only demonstrate that the MLP performed better under the tested configurations and preprocessing pipeline, rather than showing that MLPs are inherently superior to SVMs for traffic-sign recognition.

Both models achieved higher macro recall than macro precision. This means that averaged equally across the 43 classes, the models were relatively successful at identifying images belonging to each class, but their class predictions also contained a greater proportion of false positives. This difference

demonstrates why accuracy alone is insufficient for evaluating the model especially because the number of images was not evenly distributed across the traffic-sign classes. Macro-averaged precision, recall and F1-score give each class equal weighting and therefore provide additional information about performance across both large and small classes.

Performance differed across traffic-sign classes, so the overall metrics did not show how well each model performed on every class. The MLP achieved much higher F1 scores for many different classes, including Stop (0.9661 compared with 0.6557 for the SVM), Vehicles over 3.5 metric tons prohibited (0.9167 compared with 0.5333), Speed limit 20 km/h (0.7240 compared with 0.1246), and Go straight or left (0.7117 compared with 0.1688). However, the SVM performed better for several other classes, including Priority Road (0.6837 compared with 0.3618 for the MLP), Road work (0.8193 compared with 0.5837), and Yield (0.6016 compared with 0.4160). Thus although the MLP achieved stronger overall performance, neither model consistently outperformed the other across every class. Looking at the confusion matrix provides further clues into which classes were most frequently confused with one another.

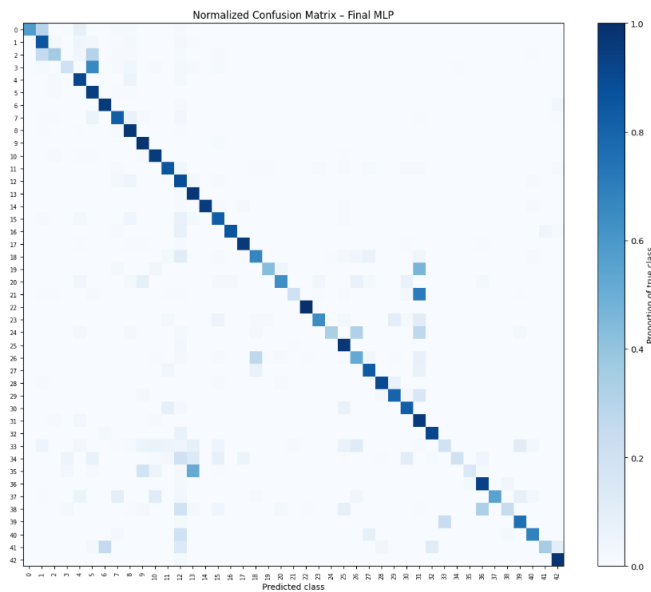


Figure 2. Row-normalized confusion matrix for the final MLP on the 16,350-image test set.

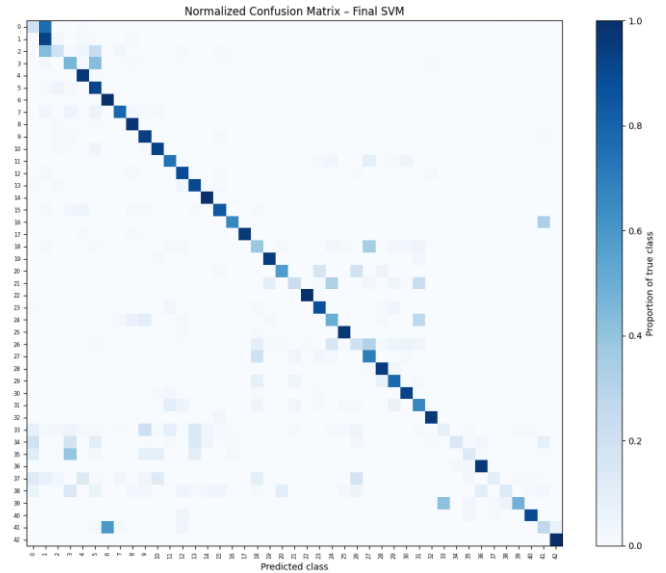


Figure 3. Row-normalized confusion matrix for the final linear SVM ($C = 1$) on the 16,350-image test set.

Rows represent the true traffic-sign class IDs and columns represent the predicted class IDs. Color intensity represents the proportion of images from each true class assigned to each predicted class.

The MLP confusion matrix showed that some visually similar signs were often mixed up. For example, about 66% of Speed limit 60 km/h images were predicted as Speed limit 80 km/h, while about 33% of Keep right images were predicted as Go straight or right. This suggests that flattening the images into pixel values made it harder for the model to keep small visual details that separate similar signs. Since each row of the confusion matrix was normalized by the true class, the diagonal values show the recall for each class. (Table with concrete numbers for each model's performance across the classes are provided in the code submission as a CSV and can also be generated from the code itself).

The normalized confusion matrices provide additional information about the class-level behavior of both models. A strong diagonal is visible in both matrices, indicating that many traffic-sign classes were identified correctly for a large proportion of their test samples. However, several classes also contain noticeable off-diagonal responses, showing that prediction errors were concentrated between particular traffic-sign classes rather than being distributed uniformly. This supports the differences observed in the per-class F1-scores. For example, the MLP achieved substantially stronger F1-scores for classes such as

Stop, Vehicles over 3.5 metric tons prohibited and Go straight or left, while the SVM performed better for classes such as Priority road, Road work and Yield. Therefore, the overall advantage of the MLP did not result from uniformly better performance across all 43 classes, but from stronger performance across a larger combination of classes.

IX. LIMITATIONS AND POSSIBLE IMPROVEMENTS

A major limitation of both models was the use of flattened RGB pixel values as the input. Flattening each $32 \times 32 \times 3$ image into a 3,072-element vector allowed the same preprocessing pipeline to be used for both models, providing a consistent comparison. However, this removed the two-dimensional spatial structure of each image. Neighbouring pixels and local shapes were therefore represented only as separate numerical features rather than as spatial patterns. This may have made it more difficult for both models to distinguish visually similar traffic signs whose differences depend on small symbols, edges or local image structure. Thus the results should be interpreted as a comparison of MLP and SVM performance using the chosen flattened-pixel representation rather than as the maximum performance achievable for traffic-sign recognition

Hyperparameter exploration was also deliberately limited by computational cost. Only three MLP configurations were evaluated, while the SVM search used a staged procedure in which all nine configurations were first screened on a subset of the training groups before the strongest candidates were evaluated using the full training partition. Consequently, the selected configurations should be interpreted as the best among those tested rather than globally optimal model configurations. A broader search over hidden-layer structures, learning rates, SVM parameters and alternative preprocessing methods could potentially improve performance.

Future work could dive into image representations that preserve spatial information more effectively – thus including image edges and surrounding pixel information. For example, additional image preprocessing, feature extraction or convolution-

based methods could be used to learn local edges, shapes and symbols before classification. These approaches may be especially useful for classes that showed repeated off-diagonal confusion in the final matrices.

X. CONCLUSION

This project compared MLP and SVM for multiclass traffic-sign recognition using the same de-duplicated and group-aware dataset. After preprocessing, hyperparameter selection and final retraining, the MLP achieved a test accuracy of 0.5149 and macro F1-score of 0.5752, compared with 0.4467 accuracy and 0.5037 macro F1 for the SVM. The MLP therefore produced stronger overall performance / scores under the tested configurations. However class-level results showed that the SVM remained more effective for certain individual traffic-sign classes. The experiment also showed that lower training loss did not necessarily correspond to better validation performance, and that careful separation of training, validation and test data was important for obtaining a more reliable estimate of generalization. Overall, the results show that model architecture, hyperparameter selection and image representation all had a significant influence on classification performance.

XI. REFERENCES

- [1] flo2607, “Traffic Signs Classification,” *Kaggle*. [Online]. Available: <https://www.kaggle.com/datasets/flo2607/traffic-signs-classification>.
- [2] Scikit-learn developers, “MLPClassifier,” *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.neural_network.MLPClassifier.html.
- [3] Scikit-learn developers, “SVC,” *scikit-learn documentation*. [Online]. Available: <https://scikit-learn.org/stable/modules/generated/sklearn.svm.SVC.html>.
- [4] Scikit-learn developers, “GridSearchCV,” *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/generated/sklearn.model_selection.GridSearchCV.html.
- [5] Scikit-learn developers, “Cross-validation: evaluating estimator performance,” *scikit-learn documentation*. [Online]. Available: https://scikit-learn.org/stable/modules/cross_validation.html.